Termination

- Termination guarantees soundness
- Termination proofs justify induction schemes
- New users are immediately confronted with termination
- CCG termination analysis: highly automated
- CCG inspired termination analyses in PVS, Isabelle

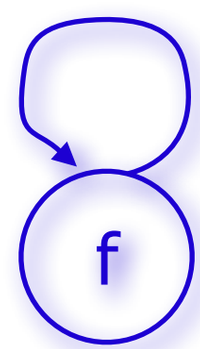
Termination

- Common recursion schemes guarantee termination
- For freshmen, failure implies non-termination
- In such cases, ACL2s provides a *termination core*
- **DEMO**

Calling Context Graphs

- Non-termination in our domain iff
- ∃ input that leads to an infinite sequence of function calls
- Goal: Conservative, precise, analyzable abstraction
- First attempt: use call graphs
- Example: Assume x, y are integers

```
f(x, y) =
if x < y
   then 1 + f(x+1, y)
   else if x > y
           then 1 + f(x, y+1)
           else 0
```



Not enough

Governors

- The governors of e’ ⊆ e are the branching conditions that need to be true for execution of e to lead to the execution of e’
- Example

```
f(x, y) =
 if x < y
    then 1 + f(x+1, y)
    else if x > y
            then 1 + f(x, y+1)
            else 0
```

- Governors for `f(x+1, y): {x < y}`

Governors

- The governors of e’ ⊆ e are the branching conditions that need to be true for execution of e to lead to the execution of e’
- Example

```
f(x, y) =
 if x < y
    then 1 + f(x+1, y)
    else if x > y
            then 1 + f(x, y+1)
            else 0
```

- Governors for `f(x+1, y): {x < y}`
- Governors for `f(x, y+1): {not(x < y), x > y}`

Calling Contexts

- A calling context for a call e is a triple containing:
  - The name of the function containing e
  - The governors for e in the function body
  - The call, e
- Example:

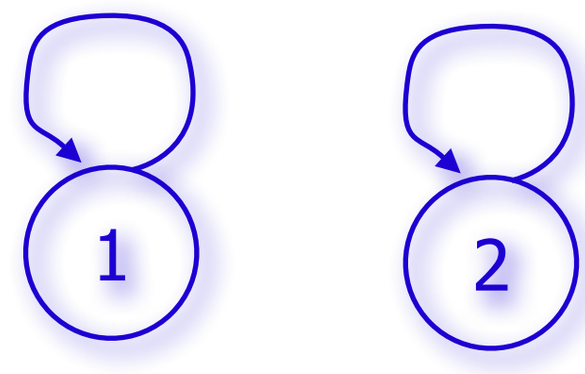
```
f(x, y) =
if x < y
   then 1 + f(x+1, y)
   else if x > y
          then 1 + f(x, y+1)
          else 0
```

1. ⟨f, {x<y}, f(x+1, y)⟩
2. ⟨f, {not(x<y), x>y}, f(x, y+1)⟩

Calling Context Graphs

- Example

```
f(x, y) =
if x < y
   then 1 + f(x+1, y)
   else if x > y
      then 1 + f(x, y+1)
      else 0
```

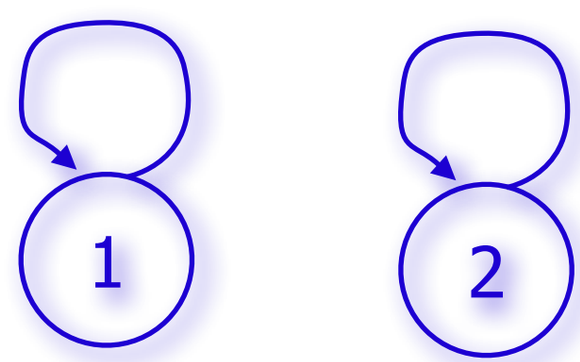


1. ⟨f, {x<y}, f(x+1, y)⟩
2. ⟨f, {not(x<y), x>y}, f(x, y+1)⟩

- Vertices are calling contexts
- An edge from c1 to c2 if it is possible for execution to reach c1 and c2 in consecutive recursive calls
- Set of all paths is an overapproximation of recursive behavior

Building Calling Context Graphs

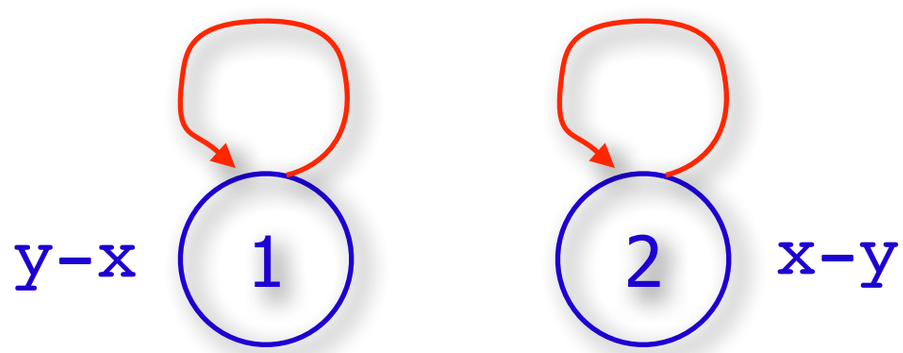
- The edge condition:
- ∃ values that satisfy the governors of both contexts
- Governors are arbitrary predicates
- Building a minimal CCG is undecidable
- Theorem prover queries used to eliminate edges
- Edge included if we cannot disprove the edge condition
- We need more



Calling Context Measures

- Map function formals into some well-founded structure
- Each calling context is given a set of CCMs
- Example:

1. ⟨f, {x<y}, f(x+1, y)⟩
2. ⟨f, {not(x<y), x>y}, f(x, y+1)⟩

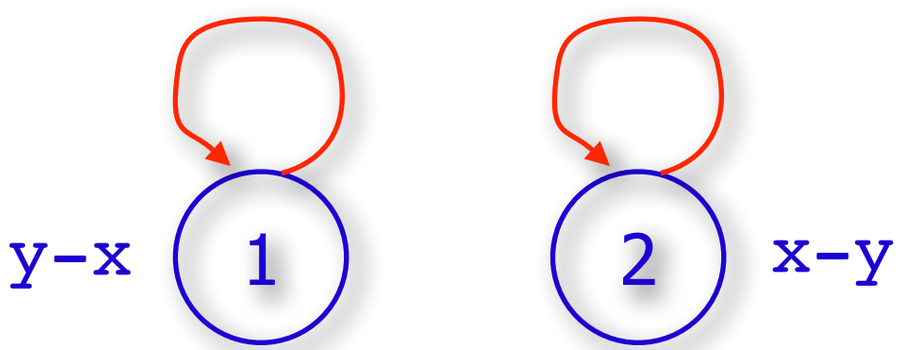


Decreasing edge >:

Non-increasing edge ≥:

The Termination Condition

- For every infinite path through the CCG,
- There should exist a corresponding sequence of CCMs,
- Such that for some tail of the sequence, each adjacent pair of CCMs is never increasing and infinitely decreasing
- Solved by Size Change back-end algorithm

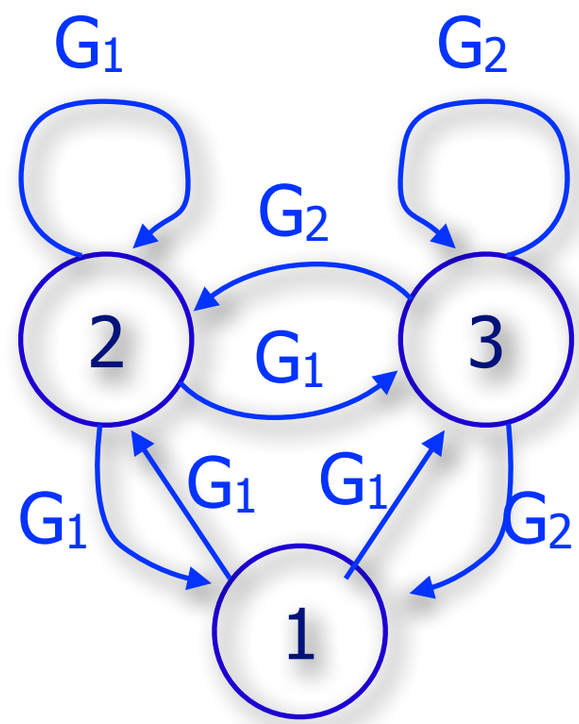


Decreasing edge >:

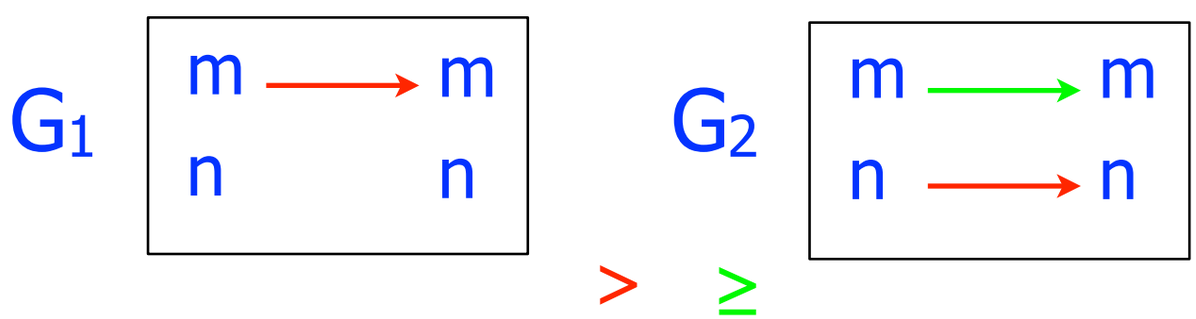
Non-increasing edge ≥:

Ackermann's Function

```
ack(m, n) =
if (zp m)
  then n+1
  else
  if (zp n)
    then ¹ack(m-1, 1)
    else ²ack(m-1, ³ack(m, n-1))
```

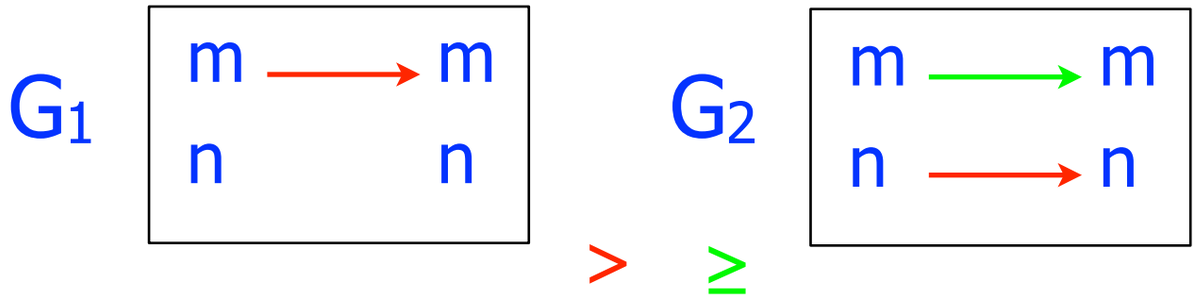


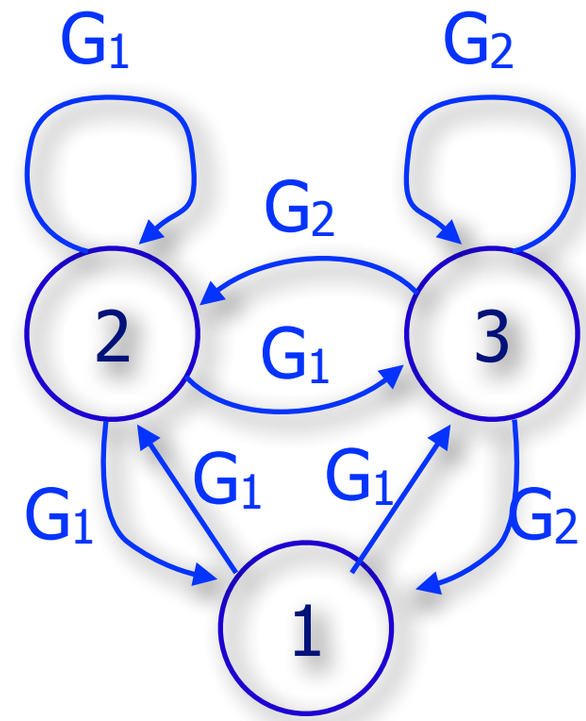
CCG graph constructed



1. ⟨ack, {natp(m), 0<m, zp(n)}, ack(m-1, 1)⟩
2. ⟨ack, {natp(m), 0<m, natp(n), 0<n}, ack(m-1, ack(m, n-1))⟩
3. ⟨ack, {natp(m), 0<m, natp(n), 0<n}, ack(m, n-1)⟩

Size Change Termination





- G_1, G_2 are generalized size change graphs (GSCGs)
- On the right is an annotated CCG (ACCG)
- A thread in a path of an ACCG is a sequence of edges
  - $\langle x_{i-1} \xrightarrow{r_i} x_i \rangle$ s.t. $x_{i-1} \xrightarrow{r_i} x_i \in G_i$ for all $i>0$
  - Eg, the path $2 \xrightarrow{G_1} 3 \xrightarrow{G_2} 1$ has one thread: $m \xrightarrow{>} m \xrightarrow{\geq} m$
- SCT: Does every infinite path in an ACCG have a suffix with an infinitely decreasing thread?
- SCT is a PSPACE-complete problem [Lee et al.]
- Algorithm: all idempotent GSCGs in cl(ACCG) have > self edge

Ackermann

Dear Pete,

Thanks for your previous answer. We made additional tests and in fact the termination method is truly very powerful. ...

In addition, some versions of Ackermann like the one below fail:

```
(defun myAck (m n)
  (if (and (integerp m) (integerp n) (>= m 0) (>= n 0))
      (if (= m 0) (+ n 1)
        (if (= n 0) (myAck (- m 1) 1)
            (myAck (myAck (- m 1) n) (- n 1))))
       nil))
```

Size-Change Cores

- Termination cores are constructed from Size Change Cores
- SCC: a finite loop, π, in an ACCG st π^ω has no suffix with a thread of infinite descent
- We construct **minimal** function definitions from cores
- Size Change Core (SCC) problem: given ACCG, either prove termination or return a SCC
- Theorem: SCC is PSPACE-complete
- A practical algorithm appears in the paper
- Same complexity as the SCT algorithm (closure)

Termination Cores

- Let T be a termination analysis, F a set of function definitions that T does not prove F terminating, and let G=(C,E) be an ACCG for F. A termination core for F modulo T is a set of function definitions F′ s.t.
  - The trivial CCG, G′=(C′, E′) of F′ is a simple cycle
  - G′ contains a subset of the recursive behaviors of G
    (∃ a similarity-preserving path homomorphism from G′ to G)
  - T does not prove F′ terminating

Full Algorithm

- We presented a simplified version of our analysis
- Mutual Recursion
- SCC analysis: can have more SCCs than functions
- Hierarchical Analysis
- Merging on a per-node basis: different edges correspond to different numbers of steps
- Multiple CCMs
- CCMs that combine formals (e.g., x-y)
- Different CCMs for different nodes
  - Saturation algorithm for propagating CCMs

JVM Example

```
(mutual-recursion
 ;; mma2 :: num, counts, s, ac --> [refs]
 (defun mma2 (c1 c2 s ac)
   (declare (xargs :measure (cons (len (cons c1 c2))
                                  (natural-sum (cons c1 c2)))))
   (if (zp c1)
       (mv (heap s) ac)
     (mv-let (new-addr new-heap)
             (mma c2 s)
             (mma2 (- c1 1)
                   c2
                   (make-state (thread-table s)
                               new-heap
                               (class-table s))
                   (cons (list 'REF new-addr) ac)))))

 ;; mma :: [counts], s --> addr, new-heap
 (defun mma (counts s)
   (declare (xargs :measure (cons (+ 1 (len counts))
                                  (natural-sum counts))))
   (if (<= (len counts) 1)

       ;; "Base case"  Handles initializing the final dimension
       (mv (len (heap s))
           (bind (len (heap s))
                 (makearray 'T_REF
                            (car counts)
                            (init-array 'T_REF (car counts))
                            (class-table s))
                 (heap s)))
     ;; "Recursive Case"
     (mv-let (heap-prime lst-of-refs)
             (mma2 (car counts)
                   (cdr counts)
                   s
                   nil)
             (let* ((obj (makearray 'T_REF
                                    (car counts)
                                    lst-of-refs
                                    (class-table s)))
                    (new-addr (len heap-prime))
                    (new-heap (bind new-addr obj heap-prime)))
               (mv new-addr new-heap)))))
 )
```

JVM Example

```
(mutual-recursion
  (defun mma2 (c1 c2 s ac)
    (if (zp c1)
        ...
        (mv-let (new-addr new-heap)
                (mma c2 s)
                (mma2 (- c1 1) c2 e e'))))

  (defun mma (c s)
    (if (<= (len c) 1)
        ...
        (mma2 (car c) (cdr c) s nil)
        ...)))

mma2 measure: (cons (len (cons c1 c2))
                    (natural-sum (cons c1 c2)))
mma measure: (cons (+ 1 (len c)) (natural-sum c))
```

Termination Expert Example

- ... generated these functions by applying a weakest precondition algorithm to 3 nested loops... This problem naturally arises when reasoning about low level programs

```
(defun f1 (w r z s x y a b zs)
  (if (not (zp z))
      (f2 w r z 0 r w 0 0 zs)
    (= w (expt r zs))))

(defun f2 (w r z s x y a b zs)
  (if (not (zp x))
      (f3 w r z s x y y s zs)
    (f1 s r (1- z) 0 0 0 0 0 zs)))

(defun f3 (w r z s x y a b zs)
  (if (not (zp a))
      (f3 w r z s x y (1- a) (1+ b) zs)
    (f2 w r z b (1- x) y 0 0 zs)))
```

- Exercise: construct a measure function

Termination Expert Example

- I looked long and hard at admitting this with ACL2 ...
- ... was not able to admit such a function, so I spent perhaps 4-6 hours ..., before convincing myself that it likely could not be done
- This set of mutually recursive definitions is admitted automatically using ACL2s
- Maybe using ACL2s is the answer
- **DEMO**

Experimental Results

- Implemented in ACL2s
- Ran our algorithm over the ACL2 Regression Suite
  - Over 100MB, 11,000 function definitions
- Ignored all user hints and other explicit assistance
- Over 98% success rate

Problems	Total	ACL2s	ACL2
Non-Trivial	1762	1544 (87%)	1056 (67%)
Recursive	4348	3394 (95%)	3642(87%)

Extensions (Projects)

- Concrete non-terminating counterexamples
  - Example: -1, -2, -3, ... or ... (a ...)*
- Proofs of nontermination
  - Extend CCG to record more information
  - Try to prove that certain cycles are feasible
  - Use cgen? to find termination counterexamples
- Integration into induction (including mutual recursion)